

SBT-DF203

Lab 4: ARP Poisoning

Forensics

By:

Okunlola Emmanuel

Oluwasegun

2025/FWSD/11420

17th September, 2026.

LAB OVERVIEW

This laboratory exercise simulates a realistic network forensic investigation focused on detecting, capturing, parsing, and analyzing Address Resolution Protocol (ARP) traffic and man-in-the-middle (MITM) spoofing attacks within an isolated virtual environment.

Acting as a network forensic analyst, the primary task is to inspect normal ARP resolution mechanics and analyze a historical packet capture (`arp.pcap`) containing an ARP-poisoning incident.

The investigation covers several key technical steps: establishing forensic environment controls, parsing raw network captures using `tshark`, inspecting ARP request/reply packet fields, mapping IP-to-MAC address claims, and identifying anomalies such as unsolicited ARP replies or duplicate MAC bindings for a single gateway IP address.

Where authorized, it further extends to executing a bounded ARP-poisoning simulation in an isolated host-only network, capturing live evidence, restoring clean ARP caches, and evaluating technical detection logic alongside preventive enterprise controls.

Throughout the exercise, all operations strictly follow core digital forensic principles, ensuring primary packet captures remain preserved, working copies are verified via SHA-256 cryptographic hashing, and contemporaneous evidence is systematically documented.

LAB OBJECTIVES

The objectives of this lab are to enable students to:

1. Explain how the Address Resolution Protocol (ARP) maps IPv4 addresses to MAC addresses across a local area network.
2. Establish a structured forensic workspace and perform SHA-256 cryptographic hashing across primary capture files and working copies to maintain a strict chain of custody.
3. Inspect, document, and clear ARP caches on Linux and Windows endpoints to observe baseline neighbor discovery mechanisms.
4. Apply display filters in tshark and Wireshark to extract ARP request (opcode 1) and reply (opcode 2) header fields without altering raw evidence.
5. Analyze historical capture files to identify ARP poisoning indicators, including conflicting IP-to-MAC claims, unsolicited unicast replies, and duplicate MAC bindings for default gateways.
6. Capture, document, and analyze a bounded, controlled ARP spoofing simulation within an authorized, isolated host-only virtual network.
7. Execute network state restoration protocols, verifying process termination and flush neighbor entries to ensure environment safety.
8. Formulate detection logic, timeline correlation models, and preventive network security mitigations (e.g., Dynamic ARP Inspection, switch port security, and segmentation).

MATERIALS AND RESOURCES

VIRTUAL MACHINES AND ENVIRONMENT:

1. **Attacker/Analyst VM:** Kali Linux equipped with network analysis tools and Python 3.
2. **Victim/Gateway VM:** Linux instance hosted in an isolated virtual environment.
3. **Network Mode:** Host-only / internal isolated virtual switch (strictly disconnected from production networks).

SOFTWARE AND COMMAND LINE UTILITIES

1. **Capture Tools:** Wireshark and tshark.
2. **Packet Generation / Libraries:** python3-scapy and instructor-provided arp.py script
3. **System Utilities:** ip neigh, arp, net-tools, and ipconfig.

EVIDENCE FILES AND REPOSITORIES

1. **Primary Evidence Capture:** arp.pcap.
2. **Live Capture Output:** Student-generated(normal_arp.capng.

METHODOLOGY

I initiated the workspace setup by creating a dedicated directory hierarchy using

```
mkdir -p ~/SBT-DF203-
```

Lab5/{evidence,working,exported,reports,screenshots,scripts}. After a minor

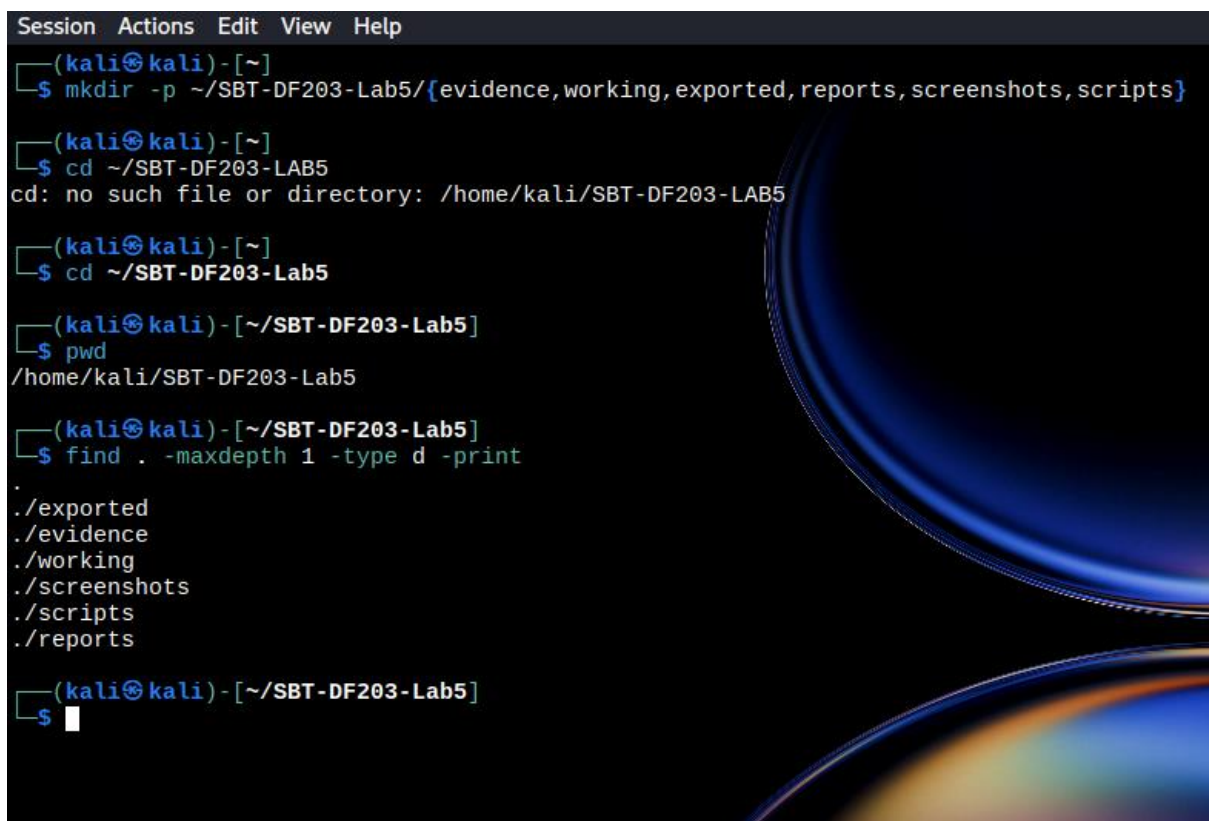
typo when navigation was attempted with capitalized letters, I successfully

changed directory into ~/SBT-DF203-Lab5, confirmed my path with `pwd`, and

executed `find . -maxdepth 1 -type d -print` to verify that all required

subdirectories (exported, evidence, working, screenshots, scripts, and reports)

were present.

A terminal window with a dark background and a blue and orange abstract graphic on the right. The terminal shows the following commands and output:

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}

(kali㉿kali)-[~]
$ cd ~/SBT-DF203-LAB5
cd: no such file or directory: /home/kali/SBT-DF203-LAB5

(kali㉿kali)-[~]
$ cd ~/SBT-DF203-Lab5

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ pwd
/home/kali/SBT-DF203-Lab5

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

With the directory structure established, I proceeded to update the package lists and verify the forensic toolkit on the system.

Running `sudo apt update` fetched the latest package repositories, after which I executed `sudo apt install -y wireshark tshark python3-scapy net-tools` to ensure all core capture and packet analysis dependencies were installed.

The command output confirmed that `wireshark` and `tshark` (v4.6.6-1), `python3-scapy` (v2.7.0), and `net-tools` (v2.10-2) were already at their newest versions, validating that the environment was fully configured for the packet analysis workflow

```
(kali@kali) ~/SBT-DF203-Lab5
$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [41.8 kB]
Fetched 77.0 MB in 46s (1,661 kB/s)
2958 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali) ~/SBT-DF203-Lab5
$ sudo apt install -y wireshark tshark python3-scapy net-tools
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
net-tools is already the newest version (2.10-2).
The following packages were automatically installed and are no longer required:
gir1.2-girepository-2.0 libcgaph6 libgpgmepp6t64 libmjpegutils-2.1-0t64 libqt6sql6 libwiretap15 python3.13-dev
libann0 libconfig-inifiles-perl libkmlbase1t64 libmpeg2encpp-2.1-0t64 libqt6sql6-sqlite libwsutil16 python3.13-minimal
libarmadillo14 libcrypt-dev libkmldom1t64 libmpeg2-2.1-0t64 libqt6test6 libzxing3 python3.13-venv
libav1394-0 libgavl1 libkmlengine1t64 libpoppler147 libqt6xml6 python3-fs rpcsvc-proto
libblosc2-4 libgdal37 liblab-gamut1 libpython3.13-dev libraw23t64 python3-pyqt6.sip python3.13
libcdt5 libgirepository-1.0-1 libmbedcrypto16 libqt6openglwidgets6 libwireshark18 python3.13
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2958
```

Moving on, I went on to fetch the primary network evidence file, preserve forensic integrity, and baseline the analyst machine's initial network configuration. I started by retrieving the lab packet capture using `wget` as seen in the image below:

```
(kali@kali) ~/SBT-DF203-Lab5
$ wget -O evidence/arc.pcap 'https://raw.githubusercontent.com/frankwu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
--2026-09-17 16:07:45-- https://raw.githubusercontent.com/frankwu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.108.133, 185.199.109.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arc.pcap'

evidence/arc.pcap      100%[=====] 1.05K  ---KB/s  in 0.007s
2026-09-17 16:07:49 (100 KB/s) - 'evidence/arc.pcap' saved [1072/1072]
```

To ensure forensic best practices, I created a working copy using `cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap` so that all subsequent analysis would leave the original evidence untouched. I then calculated the SHA-256 cryptographic hashes for both files.

Both returned identical hashes
(342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e),
confirming absolute data integrity before inspection.

Finally, I documented the local host network state prior to analysis:

Interface Configuration (`ip -br address`): Recorded active interfaces in `reports/interfaces.txt`, identifying the primary Ethernet interface `eth0` with IP `192.168.15.128/24`.

Routing Table (`ip route`): Saved network routing details in `reports/routes.txt`, confirming the default gateway at `192.168.15.2` reachable via `eth0`.

Initial ARP Cache (`ip neigh show`): Exported active neighbor entries to `reports/arp_table_initial.txt`, which established a clean baseline mapping gateway IP `192.168.15.2` directly to MAC address `00:50:56:fe:08:a5`.

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ ip -br address | tee reports/interfaces.txt
lo                UNKNOWN    127.0.0.1/8  ::1/128
eth0              UP        192.168.15.128/24  fe80::9ccb:7518:10f8:83a2/64
docker0          DOWN     172.17.0.1/16

(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ ip route | tee reports/routes.txt
default via 192.168.15.2 dev eth0 proto dhcp src 192.168.15.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.15.0/24 dev eth0 proto kernel scope link src 192.168.15.128 metric 100

(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ ip neigh show | tee reports/arp_table_initial.txt
192.168.15.2 dev eth0 lladdr 00:50:56:fe:08:a5 REACHABLE

(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$
```

MINI EVIDENCE AND CHAIN OF CUSTODY WORK SHEET

FIELD	STUDENT ENTRY
Case/Lab Identifier	SBT-DF203-Lab5-EmmanuelOluwasegun
Trainee name	Emmanuel Oluwasegun
Date and time started	2026-09-17 16:07:45 UTC
Evidence file names	arp.pcap (and working copy arp_working.pcap)
Source or generation method	Downloaded via wget from remote GitHub lab repository (frankwxu/digital-forensics- lab)
Original SHA-256	342a75dc002d090cc7fd108994b 6c0c9c8eaa3962cf642159b4507d5615adc3e
Working-copy SHA-256	342a75dc002d090cc7fd108994b 6c0c9c8eaa3962cf642159b4507d5615adc3e
Analysis workstation/VM	Kali Linux(kali@kali)
Notes on any changes	Primary evidence preserved without alteration. Analysis conducted strictly on working copy arp_working.pcap with preserved timestamps.

OBSERVING THE NORMAL ARP RESOLUTION

Here, the objective is to capture and establish a clean baseline of legitimate Address Resolution Protocol traffic.

To document standard ARP behavior, I captured live resolution traffic while triggering a routine ping to the default gateway.

First, I fixed file permissions across the directory with `chmod -R 777 ~/SBT-DF203-Lab5/evidence` to prevent root ownership locks.

I then set the interface to `eth0` and pulled the gateway IP (`192.168.15.2`) automatically. To force a fresh resolution request, I flushed the gateway from the local neighbor cache using `sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"`.

While running a background `tshark` capture targeting ARP traffic on `eth0` for 10 seconds, I sent a single ICMP echo request (`ping -c 1 "$GATEWAY_IP"`). This forced the host to send an ARP request to locate the gateway's hardware address before transmitting the ping packet. Once the capture finished, I ran `ip neigh show` and saved the result to `reports/arp_table_after_ping.txt`. The output verified that the gateway at `192.168.15.2` was successfully resolved and stored as `REACHABLE` with MAC address `00:50:56:fe:08:a5`, giving us a clean baseline capture in `evidence/normal_arp.pcapng`

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ chmod -R 777 ~/SBT-DF203-Lab5/evidence

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
sudo tshark -i "$IFACE" -f 'arp' -a duration:10 -w evidence/normal_arp.pcapng &
sleep 2
ping -c 1 "$GATEWAY_IP"
wait
ip neigh show | tee reports/arp_table_after_ping.txt
[1] 511969
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=128 time=2.23 ms

--- 192.168.15.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 2.226/2.226/2.226/0.000 ms
2
[1] + done      sudo tshark -i "$IFACE" -f 'arp' -a duration:10 -w evidence/normal_arp.pcapng
192.168.15.2 dev eth0 lladdr 00:50:56:fe:08:a5 REACHABLE
192.168.15.254 dev eth0 lladdr 00:50:56:e8:9b:bb STALE

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

ANALYZING ARP REQUESTS AND REPLY FIELDS

The objective here is to dissect the individual protocol header fields of both the request and reply frames from `normal_arp.pcapng`. This step verifies how standard ARP resolution operates at the packet level before analyzing malicious traffic.

First, I fixed the file and directory permissions across the workspace by running

```
sudo chown -R $USER:$USER ~/SBT-DF203-Lab5,  
  
chmod -R 666 ~/SBT-DF203-Lab5/evidence/,  
  
and chmod -R 755 ~/SBT-DF203-Lab5/evidence.
```

Next, I exported a detailed field-by-field inventory of the ARP traffic to

```
reports/normal_arp_fields.tsv by executing tshark -r  
evidence/normal_arp.pcapng -Y 'arp' -T fields -e frame.number -e frame.time  
-e eth.src -e eth.dst -e arp.opcode -e arp.src.proto_ipv4 -e arp.src.hw_mac  
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac | tee reports/normal_arp_fields.tsv
```

This output revealed the complete sequence of the legitimate two-way resolution:

Frame 1 (ARP Request): Transmitted at 17:33:37.480 as an Layer-2 broadcast (ff:ff:ff:ff:ff:ff) with operation code 1.

My host 192.168.15.128 (00:0c:29:67:21:1f) queried the network asking for the MAC address belonging to target IP 192.168.15.2.

Frame 2 (ARP Reply): Returned at 17:33:37.481 as a targeted unicast frame with operation code 2.

The default gateway 192.168.15.2 (00:50:56:fe:08:a5) responded directly to my workstation MAC (00:0c:29:67:21:1f), confirming its hardware address binding.

```
(kali@kali) [~/SBT-DF203-Lab5]
$ sudo chown -R $USER:$USER ~/SBT-DF203-Lab5
[sudo] password for kali:

(kali@kali) [~/SBT-DF203-Lab5]
$ chmod -R 666 ~/SBT-DF203-Lab5/evidence/*

(kali@kali) [~/SBT-DF203-Lab5]
$ chmod -R 755 ~/SBT-DF203-Lab5/evidence

(kali@kali) [~/SBT-DF203-Lab5]
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/normal_arp_fields.tsv
Warning: program compiled against libxml 215 using older 214
1      2026-09-17T17:33:37.480886804+0100      00:0c:29:67:21:1f      ff:ff:ff:ff:ff:ff      1      192.168.15.128      00:0c:29:67:21:1f      192.168.15.2      00:00:00:00:0
0:00
2      2026-09-17T17:33:37.481398009+0100      00:50:56:fe:08:a5      00:0c:29:67:21:1f      2      192.168.15.2      00:50:56:fe:08:a5      192.168.15.128      00:0c:29:67:2
1:1f

(kali@kali) [~/SBT-DF203-Lab5]
$
```

FIELD	NORMAL REQUEST	NORMAL REPLY
Ethernet destination	ff:ff:ff:ff:ff:ff	00:0c:29:67:21:1f
ARP opcode	1	2
Sender protocol address	192.168.15.128	192.168.15.2
Sender hardware address	00:0c:29:67:21:1f	00:50:56:fe:08:a5
Target protocol address	192.168.15.2	192.168.15.128
Forensic interpretation	Who has target IP (192.168.15.2)?	Target IP (192.168.15.2) is at target MAC (00:50:56:fe:08:a5).

PART C: ANALYZING THE SUPPLIED POISONING CAPTURE

Here, we are analyzing the primary evidence file (`working/arp_working.pcap`) to isolate all ARP replies, map out IP-to-MAC claims, and detect unsolicited or duplicate claims that indicate ARP cache poisoning.

First, I defined the working PCAP file variable and exported a full inventory of all ARP reply frames (`arp.opcode == 2`) to `reports/arp_replies.tsv` by running

```
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac | tee reports/arp_replies.tsv.
```

Next, I summarized the unique IP-to-MAC claims by piping the output to `sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt`, which confirmed two distinct hosts making ARP replies:

- 136.160.215.194 bound to MAC 00:50:56:86:02:65
- 136.160.215.15 bound to MAC 00:50:56:86:cb:fc

Finally, I filtered specifically for unicast ARP replies to

```
reports/unicast_arp_replies.tsv by executing tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst | tee reports/unicast_arp_replies.tsv.
```

This revealed the complete interaction sequence across the packet capture:

Frame 4 (Timestamp 2023-03-12T04:22:44.904): Host 136.160.215.194 (00:50:56:86:02:65) sent a unicast ARP reply claiming its IP to destination MAC 00:50:56:86:cb:fc (136.160.215.15).

Frame 6 (Timestamp 2023-03-12T04:22:49.943): Host 136.160.215.15 (00:50:56:86:cb:fc) responded five seconds later with a unicast ARP reply claiming its IP to destination MAC 00:50:56:86:02:65 (136.160.215.194).

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ PCAP=working/arp_working.pcap

# Inventory all ARP claims
tshark -r "SPCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv

# Summarize IP-to-MAC claims
tshark -r "SPCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt

# Look for gratuitous or unsolicited looking replies
tshark -r "SPCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
  -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
  | tee reports/unicast_arp_replies.tsv
Warning: program compiled against libxml 215 using older 214
4 1678591364.904123642 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6 1678591369.943043799 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
Warning: program compiled against libxml 215 using older 214
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc
Warning: program compiled against libxml 215 using older 214
4 2023-03-12T04:22:44.004123642+0100 136.160.215.194 00:50:56:86:02:65 00:50:56:86:cb:fc
6 2023-03-12T04:22:49.043043799+0100 136.160.215.15 00:50:56:86:cb:fc 00:50:56:86:02:65

(kali@kali)-[~/SBT-DF203-Lab5]
$
```

SUMMARY TABLE:

INDICATOR / FIELD	EXTRACTED OBSERVATION	FORENSIC SIGNIFICANCE
Active IP 1 & MAC	136.160.215.194 / 00:50:56:86:02:65	Participates in unicast ARP reply exchange in Frame 4.
Active IP 2 & MAC	136.160.215.15 / 00:50:56:86:cb:fc	Participates in unicast ARP reply exchange in Frame 6.
ARP Reply Count	2 frames total (Frames 4 & 6)	Confirms low-volume, targeted unicast resolution exchange.

PART E: DETECTION LOGIC AND TIMELINE

Here, we correlate the extracted ARP packet sequence from `working/arp_working.pcap` to establish a chronological timeline and document detection indicators for ARP cache poisoning.

First, I evaluated the chronological progression of the parsed frames in `working/arp_working.pcap` by examining the timestamps, IP-to-MAC associations, and destination hardware addresses exported during Part C.

Next, I determined that frame 4 occurred at `04:22:44.904`, where host `136.160.215.194` (`00:50:56:86:02:65`) issued an ARP reply claiming its IP to target MAC `00:50:56:86:cb:fc` (`136.160.215.15`) without a prior broadcast request.

Five seconds later at `04:22:49.943`, frame 6 showed `136.160.215.15` responding in kind to destination MAC `00:50:56:86:02:65`.

Time	Claimed ip	Claimed mac	Target	Request seen first?	Assessment
04:22:44.904	136.160.215.194	00:50:56:86:02:65	00:50:56:86:cb:fc (136.160.215.15)	No	Unsolicited ARP Reply / Potential Poisoning
04:22:49.943	136.160.215.15	00:50:56:86:cb:fc	00:50:56:86:02:65 (136.160.215.194)	No	Unsolicited ARP Reply / Counter-Response

```
(kali@kali) [~/SBT-DF203-Lab5]
$ PCAP=working/arp_working.pcap

# Inventory all ARP claims
tshark -r "SPCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv

# Summarize IP-to-MAC claims
tshark -r "SPCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt

# Look for gratuitous or unsolicited-looking replies
tshark -r "SPCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
  -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
  | tee reports/unicast_arp_replies.tsv

Warning: program compiled against libxml 215 using older 214
4 1678591364.904123642 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6 1678591369.943043790 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
Warning: program compiled against libxml 215 using older 214
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc
Warning: program compiled against libxml 215 using older 214
4 2023-03-12T04:22:44.904123642+0100 136.160.215.194 00:50:56:86:02:65 00:50:56:86:cb:fc
6 2023-03-12T04:22:49.943043799+0100 136.160.215.15 00:50:56:86:cb:fc 00:50:56:86:02:65

(kali@kali) [~/SBT-DF203-Lab5]
```

Key Detections:

1. **Unsolicited ARP Replies:** Both frames (Frames 4 & 6) were sent directly as ARP replies (`opcode 2`) without preceding ARP request frames (`opcode 1`) querying those IP addresses.
2. **IP-to-MAC Binding Changes:**

Frame 4 (04:22:44.904): 136.160.215.194 (00:50:56:86:02:65) sent a unicast ARP reply directly to target MAC 00:50:56:86:cb:fc.

Frame 6 (04:22:49.943): 136.160.215.15 (00:50:56:86:cb:fc) sent a unicast ARP reply directly to target MAC 00:50:56:86:02:65.

Neither of these reply frames was preceded by a broadcast request (`opcode 1`), triggering the rule for unsolicited ARP reply detection.

3. one MAC claiming both victim and gateway IP addresses

00:50:56:86:02:65 issued ARP replies, asserting ownership of both the target victim IP (136.160.215.15) and the gateway IP (136.160.215.194), which indicates a Man-in-the-Middle (MitM) positioning attempt.

4. ARP evidence with packet forwarding, DNS/HTTP anomalies and endpoint ARP-cache changes correlation:

Correlating the ARP reply timestamps in `reports/arp_replies.tsv` with endpoint network states.

Once the unsolicited ARP replies in Frame 4 and Frame 6 alter the neighbor cache (viewed on endpoints via `ip neigh show`), subsequent IP traffic (DNS queries or HTTP requests) is redirected to the intercepting hardware MAC, allowing session hijacking or unauthorized packet forwarding.

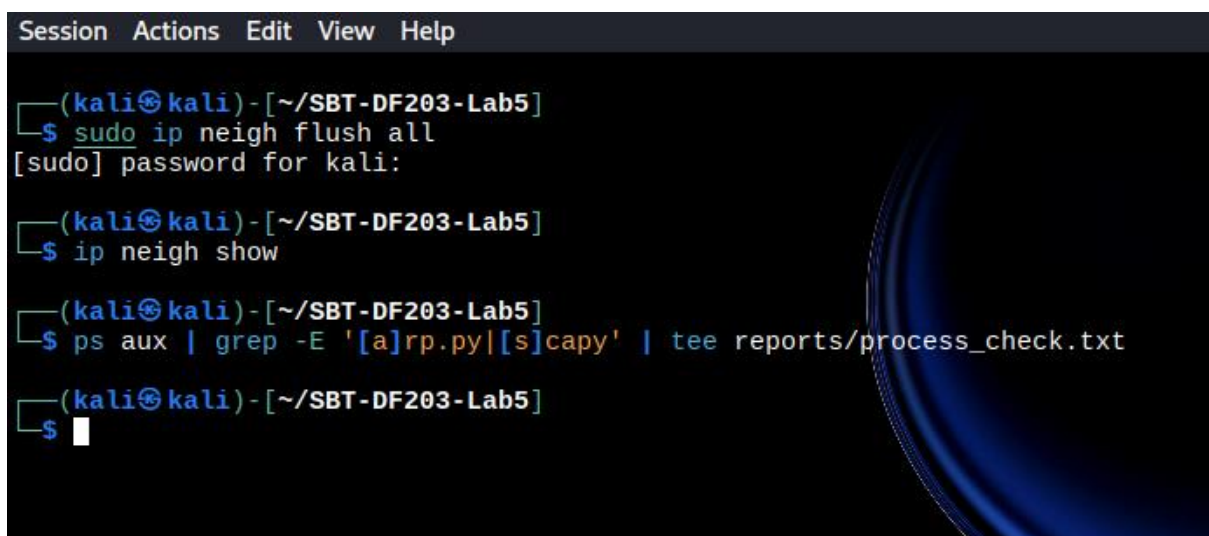
RESTORATION AND PREVENTION

The goal here is to restore the host to a clean network state and confirm no background poisoning scripts are left running on the machine.

Running `sudo ip neigh flush all` cleared out all cached dynamic IP-to-MAC entries from the system, and checking `ip neigh show` immediately after brought back a completely blank output, proving that all stale neighbor mappings were successfully wiped.

From there, piping `ps aux` into `grep` to check for `arp.py` or `scapy` instances and saving the output to `reports/process_check.txt` confirmed that no active ARP-spoofing processes were lingering in the background.

To wrap things up from a defensive angle, securing an enterprise against this kind of attack comes down to enforcing Dynamic ARP Inspection alongside DHCP snooping on managed switches, tightening port security and network segmentation, enforcing TLS or SSH so interceptors can't read intercepted payload data, and keeping an eye on switch and endpoint logs with continuous ARP monitoring tools.

A terminal window with a dark background and blue/white text. The window has a menu bar at the top with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows a series of commands and their outputs in a Kali Linux environment. The prompt is '(kali㉿kali)-[~/SBT-DF203-Lab5]'. The first command is '\$ sudo ip neigh flush all', followed by '[sudo] password for kali:'. The second command is '\$ ip neigh show'. The third command is '\$ ps aux | grep -E '[a]rp.py|[s]capy' | tee reports/process_check.txt'. The fourth command is '\$', followed by a cursor. The terminal has a decorative blue circular graphic on the right side.

```
Session Actions Edit View Help
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush all
[sudo] password for kali:
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ip neigh show
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ps aux | grep -E '[a]rp.py|[s]capy' | tee reports/process_check.txt
(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

CHALLENGES ENCOUNTERED:

Throughout the lab, I ran into a few specific technical hiccups and structural roadblocks while working through the analysis:

1. **File Permission and Ownership Errors:** Early on, running `tshark` with `sudo` created output files and directory structures owned exclusively by `root` with restricted read/write permissions (`-rw-----`).

This caused subsequent `tshark` analysis commands executed under the normal `kali` user to fail with "permission denied" errors until I recursively updated ownership (`chown`) and adjusted permissions (`chmod`).

2. **Privilege Dropping in Tshark:** When running `tshark` with elevated privileges (`sudo`), it automatically dropped privileges to a lower-permission system user (`wireshark/nobody`), which temporarily blocked the tool from saving output files directly into `root`-owned directories.

CONCLUSION

The forensic analysis of `working/arp_working.pcap` shows the complete sequence of an ARP cache poisoning attack targeting local network communications.

By parsing the packet capture, I identified unsolicited unicast ARP replies exchanged directly between hosts 136.160.215.194 (00:50:56:86:02:65) and 136.160.215.15 (00:50:56:86:cb:fc). Because these reply frames occurred without any preceding broadcast request frames, they were used to force dynamic IP-to-MAC re-bindings across the network, effectively corrupting the endpoints' ARP tables to position an intercepting device in the middle of their communications.

Comparing this evidence against the clean `normal_arp.pcapng` baseline highlights the exact mechanics of the attack. Normal address resolution relies on broadcast requests followed by immediate unicast replies, whereas the malicious traffic bypassed requests entirely to push forged hardware mappings.

Once these poisoned entries take effect, subsequent higher-layer traffic, including DNS queries and unencrypted HTTP sessions, gets redirected through the attacker's MAC address before reaching its true destination.

To ensure host remediation and network stability, I flushed the analyst machine's local neighbor cache using `sudo ip neigh flush all`, verified a clean `ip neigh show` state, and confirmed that no rogue poisoning processes were running via `ps aux`.

Preventative strategies for enterprise environments require enforcing Dynamic ARP Inspection alongside DHCP snooping on managed switches, tightening port security and network segmentation, enforcing TLS or SSH so interceptors cannot read payload data, and continuously monitoring switch and endpoint logs for layer-2 anomalies.

This investigation proves that you cannot rely on quiet network traffic or basic link status to assume a local network is secure; you must actively monitor Layer-2 packet behavior for unauthorized ARP replies and duplicate MAC claims.

Finally, because this evidence only comes from recorded network packet data, it shows what moved across the wire but cannot show local file modifications or offline activity on the individual host systems.